UNIVERSIDADE PAULISTA - UNIP
INSTITUTO DE CIÊNCIAS EXATAS E TECNOLOGIA
CURSO DE CIÊNCIA DA COMPUTAÇÃO / ENGENHARIA DA COMPUTAÇÃO

SISTEMA DE MONITORAMENTO AMBIENTAL DO RIO TIETÊ

Atividade Prática Supervisada (APS) apresentada como requisito parcial para obtenção de nota nas disciplinas do semestre corrente do curso de Ciência da Computação da Universidade Paulista.

Integrantes do Grupo:
• Thiago Leopoldo Paixão - RA: N0992C1

SÃO PAULO - SP
2026

1. INTRODUÇÃO E OBJETIVOS

A preservação dos recursos hídricos e a fiscalização de efluentes industriais são desafios centrais na gestão ambiental urbana. O Rio Tietê, atravessando grande parte do estado de São Paulo, sofre historicamente com o descarte irregular de poluentes químicos e orgânicos provenientes de complexos industriais e urbanos.

Este trabalho apresenta o desenvolvimento e a implementação de um sistema distribuído de monitoramento ambiental em tempo real. O objetivo é conectar equipes de fiscalização de campo (Inspetores) à central da Secretaria do Meio Ambiente por meio de uma arquitetura de rede confiável e estruturada.

2. REFERENCIAL TEÓRICO

2.1 Arquitetura Cliente-Servidor

A arquitetura Cliente-Servidor consiste em uma estrutura de aplicação distribuída que divide as tarefas entre fornecedores de recursos ou serviços, chamados servidores, e os requerentes de serviços, chamados clientes. No modelo desenvolvido, a aplicação **Secretaria (Servidor)** aguarda passivamente requisições, enquanto a aplicação **Inspetor (Cliente)** estabelece conexões ativas para envio de dados sanitários.

2.2 Protocolo de Transporte TCP/IP

Para garantir a confiabilidade na entrega dos relatórios ambientais, a camada de transporte utiliza o protocolo **TCP (Transmission Control Protocol)**. Diferente do UDP, o TCP oferece comunicação orientada à conexão, garantindo que nenhum pacote de dados seja perdido ou entregue fora de ordem, realizando a checagem de erros e confirmação de recebimento (acknowledgment).

Fundamentação dos Sockets TCP/IP

Os Sockets de Berkeley fornecem a interface de programação (API) padronizada para comunicação de rede inter-processos. Em C# (.NET 8.0), a abstração é fornecida pelas classes `TcpListener` (no lado do servidor) e `TcpClient` (no lado do cliente), operando sobre o fluxo de dados em rede (`NetworkStream`).

2.3 Serialização de Dados em JSON

A transferência de objetos estruturados entre sistemas na rede requer a conversão do estado do objeto em memória para uma sequência de bytes. Adotou-se o formato **JSON (JavaScript Object Notation)** via namespace `System.Text.Json`, permitindo a serialização e desserialização de instâncias da classe `RelatorioInspecao` com legibilidade e baixo overhead de rede.

3. ARQUITETURA E IMPLEMENTAÇÃO DO SISTEMA

O sistema foi construído em C# (.NET 8.0) utilizando Windows Forms para as interfaces gráficas. A solução foi dividida em quatro componentes essenciais para garantir o desacoplamento de código:

Componente	Papel na Arquitetura	Descrição Técnica
`SecretariaForm`	Servidor / Interface	Inicia o `TcpListener` na porta 5000, escuta conexões assíncronas e atualiza a interface gráfica.

`InspetorForm`	Cliente / Interface	Coleta dados do campo (Indústria, Local, PPM) e aciona o envio de dados.
`ClienteInspetor`	Módulo de Rede	Gerencia o socket `TcpClient`, implementa `IDisposable` e efetua o streaming do JSON.
`RelatorioInspecao`	Modelo de Dados	Classe DTO contendo as propriedades da medição e os métodos de serialização.

3.1 Implementação do Módulo de Comunicação

Abaixo é destacado o trecho essencial da transmissão assíncrona dos relatórios via Socket TCP:

```
public async Task<bool> EnviarRelatorioAsync(RelatorioInspecao relatorio) { using (TcpClient client = new TcpClient()) { await client.ConnectAsync(ipServidor, porta); using (NetworkStream stream = client.GetStream()) { string json = JsonSerializer.Serialize(relatorio); byte[] buffer = Encoding.UTF8.GetBytes(json); await stream.WriteAsync(buffer, 0, buffer.Length); return true; } } }
```

4. RESULTADOS E CONCLUSÃO

A aplicação foi validada com sucesso no ambiente Windows, demonstrando estabilidade na transmissão bidirecional de dados. O uso do padrão de programação assíncrona (`async/await`) garantiu que as interfaces do usuário permanecessem responsivas durante as operações de entrada e saída na rede.

Em suma, a Atividade Prática Supervisada permitiu consolidar os conhecimentos de redes de computadores, protocolos de transporte e programação orientada a objetos distribuída, cumprindo integralmente os requisitos acadêmicos estipulados.

5. REFERÊNCIAS BIBLIOGRÁFICAS

- KUROSE, James F.; ROSS, Keith W. **Redes de Computadores e a Internet: Uma Abordagem Top-Down**. 8. ed. São Paulo: Pearson, 2021.
- MICROSOFT. **Documentação do .NET e Windows Forms**. Disponível em: <https://learn.microsoft.com/dotnet/>. Acesso em: 2026.
- TANENBAUM, Andrew S.; WETHERALL, David. **Redes de Computadores**. 5. ed. São Paulo: Pearson, 2011.